Smart Thermostat Energy Saver

Team Members

- Jessica Salguero, UIN: 937005006
- Omar Khan, UIN: 437006822

Proposed Project Form

Practice: Reinforcement Learning System for a Smart Thermostat

What We Plan To Do & How

The goal of this project is to develop a reinforcement learning system for controlling a simulated residential smart thermostat. Throughout this implementation, we will investigate whether an RL-based system is able to learn a balance between resident comfort and electricity costs.

Our main question is: can an RL agent learn anticipatory behavior, such as pre-heating or pre-cooling ahead of occupancy and peak prices, that a reactive deadband controller cannot, and how does that behavior change as we shift the balance between energy cost and occupant comfort?

Proposed Starting Point

During this project, we plan to build the main required components using Python. We will create a single-home environment using Gymnasium and implement key features including a reward function, state discretization, a tabular Q-learning algorithm, and model evaluation. We will also be using established Python libraries for basic functionalities, such as NumPy, Pandas, and Matplotlib for data handling and visualization.

Our initial RL agent will use tabular Q-learning with an ε-greedy implementation. We will be starting this implementation from scratch rather than attempting to directly recreate an already published agent, but may use online tools for help if we come across useful resources as we begin work. Key features will include outdoor temperature, resident occupancy, desired “comfortable” temperature range, and electricity prices per kWh. Depending on the time required to implement the baseline RL agent, additional phases may include comparing its performance with a Deep Q-Network (DQN) and incorporating historical weather data into the simulation.

We do not currently require anything from the course staff in order for us to implement this project. However, if any HVAC or energy usage datasets could provide good insight and validation into the accuracy of our simulated results, please let us know.

State Space

The state space will represent the current conditions of a simulated home environment that are directly related to temperature control and thermostat performance. For our initial tabular Q-learning implementation, a state will be defined as:

- Indoor temperature (°F): temperature binned in ~0.5° increments for the tabular version; raw float for DQN
- Outside temperature (°F): binned/raw temperature
- Hour of day: either proxy for time-of-use pricing and occupancy patterns; debating whether we want a 0-23 hour implementation or bin the day into broader categories (morning, daytime/work hours, evening, night/sleep hours)
- Occupancy: binary representation of whether the resident is home or away
- Day type: weekday or weekend
- Current HVAC mode: off / heating/cooling
- Time-of-use energy price tier: off-peak / peak

Action Space

Our initial action space will be defined as:

- HVAC off
- Low heating
- High heating
- Low cooling
- High cooling

This action space can be used with both tabular Q-learning and DQN.

As a stretch goal, we may investigate a continuous action space in which the agent chooses an HVAC power value between full cooling and full heating. This would allow us to test a continuous-control algorithm such as DDPG or PPO.

Transition Function

We will implement a simplified single-zone thermal model based on a resistor–capacitor (RC) analogy commonly used in building energy simulation. The transition function will model how the indoor temperature changes over time based on the current indoor temperature, outdoor temperature, and the selected HVAC action:

$$indoor_T_{t+1} = indoor_T_t + \frac{\Delta t}{C} * [\frac{outside_T_t - indoor_T_t}{R} + Q_HVAC_{action_t}] + \varepsilon$$

Within this function, each variable and parameter represents different components related to the simulated home environment:

- R: thermal resistance or insulation (simulation parameter)
- C: thermal mass (simulation parameter)

- Q_HVAC: heating or cooling power associated with the selected action at time *t*
- ε: small amount of random variation to the simulated temperature
- indoor_T_t: indoor temperature at time *t*
- outside_T_t: outside temperature at time *t*
- Δt: length of timestep

This model allows the indoor temperature to gradually move toward the outdoor temperature while also accounting for the effect of heating or cooling actions. As a result, each HVAC action affects both the immediate indoor temperature and the states encountered during future timesteps.

Reward Function

The reward function will penalize both electricity usage and occupant discomfort. We currently plan to implement a reward function in this form:

$$Reward = -[(\alpha * E_cost) + (\beta * discomfort_penalty) + (\gamma * switch_penalty)]$$

Within this function, each variable/parameter represents different components related to the simulated home environment:

- E_cost: energy cost based on HVAC power consumption and current electricity price
- discomfort_penalty: penalty based on how far the indoor temperature falls outside of the resident's desired comfortable temperature range
- switch_penalty: penalty given each time the HVAC system switches between heating, cooling, and off. This discourages excessive switching between modes
- α, β, γ: tunable weights for controlling the importance of each component

The discomfort penalty will be weighted more heavily when the home is occupied, since maintaining a comfortable temperature is more important when residents are present. We may also assign a larger penalty when the indoor temperature moves significantly outside of the desired comfortable range. By adjusting the reward weights, we can evaluate how the learned policy changes when more importance is placed on minimizing energy cost versus maintaining occupant comfort.

Sequential Problem

We will train an RL agent to control the heating and cooling of a simulated home, while also trying to minimize energy cost and keep the indoor temperature within a comfortable range. This is a sequential problem because each thermostat decision will affect future decisions and states. For example, heating the home now may increase the immediate energy cost, but it may also reduce how much heating will be needed at a future time. However, leaving the HVAC system off would minimize energy cost but could lead to resident discomfort.

Since indoor temperature typically changes at a more gradual speed throughout the day, the agent will need to consider how current actions will affect future states, rather than only focusing

on the current temperature. This means that the agent can potentially learn behaviors that are not simply reactive and will instead need to plan ahead and consider actions such as pre-heating before an occupant returns, rather than making a single-shot decision. Reward at each timestep depends on the full trajectory of past actions through the current indoor temperature.

Simplest First Results

Our first goal will be to create the simulated environment and successfully train a tabular Q-learning agent using an ε-greedy policy.

Before training the RL agent, we will implement two comparison controllers:

- standard deadband thermostat controller: turns heating on when the indoor temperature drops below a threshold and cooling on when it rises above another threshold
- random-action agent: a lower-bound sanity check

The deadband controller will be our primary baseline because it approximates the type of simple reactive control used by conventional thermostats.

In addition to our RL reward, we will also evaluate each controller using separate metrics:

- total energy cost
- total or average comfort violation
- percentage of occupied time within the target comfort range
- number of HVAC mode changes

Our simplest successful result would be for the Q-learning controller to perform at least as well as the deadband controller while improving either energy cost or occupant comfort. For example, it may reduce energy cost while maintaining a similar comfort level, or improve comfort without greatly increasing energy use.

Planned Behavioral Comparisons

After obtaining a working Q-learning policy, we would like to compare the behavior of the RL controller with the deadband baseline under several conditions. These may include:

- different outside-temperature patterns
- weekday versus weekend occupancy
- peak versus off-peak electricity pricing
- different thermal properties representing better- or worse-insulated homes
- different weights on energy cost versus occupant comfort.

These experiments will help us compare how the RL agent behaves under different conditions and identify situations where it performs better or worse than the simpler deadband controller.

Stretch Goals

Depending on our progress, possible stretch goals include:

1. DQN comparison: Compare the tabular Q-learning agent with a Deep Q-Network using continuous state inputs and the same discrete HVAC action space.
2. Historical weather data: Replace the synthetic outside-temperature pattern with real weather data and evaluate the agent across a longer period or multiple seasons.
3. Continuous HVAC control: Replace the discrete HVAC actions with a continuous heating/cooling signal and compare a continuous-control algorithm such as DDPG or PPO.
4. Multiple thermal zones: Extend the simulator from a single room or zone to multiple connected rooms with different temperatures.
5. Uncertain occupancy: Rather than providing exact occupancy directly to the controller, introduce imperfect occupancy information or predictions and evaluate the robustness of the learned policy.
6. Long-term energy comparison: Simulate a longer period and estimate the percentage reduction in energy cost relative to the deadband controller.

These extensions would allow us to investigate how the difficulty of the RL problem changes as the environment becomes more realistic.

Implementation Plan

Our project execution is structured into sequential phases, ensuring thorough testing before proceeding. To ensure consensus on underlying mathematical formulations, we will jointly pair-program the core environment and reward function modules, after which individual components will be assigned independently once baseline interfaces stabilize.

1. Implement the RC thermal simulation model along with associated unit test suites.
2. Wrap the thermal model within a Gymnasium environment, validating reset/step dynamics and reward signaling.
3. Execute end-to-end simulation runs using baseline controllers (deadband, scheduled setback, and random agent).
4. Develop and train the tabular Q-learning agent to serve as our primary baseline result.
5. Construct unified evaluation scripts and performance metrics for standardized model comparisons.
6. Conduct behavioral comparison experiments and analyze the tradeoff relationship between energy cost and comfort.
7. Investigate secondary stretch goals in designated feature branches according to the prioritized order above.